同濟大學

TONGJI UNIVERSITY

实验报告

装

订

线

实验名称	循环神经网络（RNN）
课程名称	深度学习
学　　院	计算机科学与技术学院
专　　业	数据科学与大数据技术
学生姓名	景琛皓
学　　号	2351635
指导教师	程大伟
日　　期	2026.3.19

1 程序补全与代码修改说明

这里我们选择 pytorch 版本，需要修改 rnn.py 和 main.py

1.1 rnn.py ：LSTM 层定义

在 rnn.py 的 RNN_model.__init__ 中，补全了 self.rnn_lstm：

```
def __init__(self, batch_sz ,vocab_len ,word_embedding,embedding_dim, lstm_hidden_dim):
        super(RNN_model,self).__init__()
        self.word_embedding_lookup = word_embedding
        self.batch_size = batch_sz
        self.vocab_length = vocab_len
        self.word_embedding_dim = embedding_dim
        self.lstm_dim = lstm_hidden_dim
        ##########################################
        # here you need to define the "self.rnn_lstm"  the input size is "embedding_dim" and
the output size is "lstm_hidden_dim"
        # the lstm should have two layers, and the  input and output tensors are provided as
(batch, seq, feature)
        self.rnn_lstm = nn.LSTM(input_size=embedding_dim, hidden_size=lstm_hidden_dim,
num_layers=2, batch_first=True)
        ##########################################
        self.fc = nn.Linear(lstm_hidden_dim, vocab_len )
        self.apply(weights_init) # call the weights initial function.
        self.softmax = nn.LogSoftmax(dim=1) # the activation function.
        # self.tanh = nn.Tanh()
```

- input_size=embedding_dim ：每个时间步输入的是“字向量”（embedding），其维度为 embedding_dim 。也就是模型不是直接吃字符 ID，而是吃 embedding 后的连续向量表示。

- hidden_size=lstm_hidden_dim ：LSTM 隐状态维度，决定模型记忆容量与表达能力。维度越大，模型越能拟合复杂结构，但计算更慢、也更易过拟合。

- num_layers=2 ：堆叠两层 LSTM（双层堆叠）。第一层提取较浅层的序列模式，第二层在其基础上学习更抽象的语义/韵律结构，通常比单层更有表达力。

- batch_first=True ：输入输出张量形状约定为 (batch, seq_len, feature) ，与本工程中 view(1, -1, embedding_dim) 的组织方式匹配，减少维度混乱风险。

1.2 rnn.py ：前向传播

在 RNN_model.forward 中，补全了将 batch_input 输入 LSTM 并得到输出：

```
def forward(self,sentence,is_test = False):
        batch_input =
self.word_embedding_lookup(sentence).view(1,-1,self.word_embedding_dim)
        # print(batch_input.size()) # print the size of the input
        ################################################
        # here you need to put the "batch_input"  input the self.lstm which is defined before.
        # the hidden output should be named as output, the initial hidden state and cell state
set to zero.
        output, (hidden, cell) = self.rnn_lstm(batch_input)
        ################################################
        out = output.contiguous().view(-1,self.lstm_dim)
        out =  F.relu(self.fc(out))
```

```
        out = self.softmax(out)
        if is_test:
            prediction = out[ -1, : ].view(1,-1)
            output = prediction
        else:
           output = out
        # print(out)
        return output
```

- batch_input 是把输入的字符索引先过 embedding，再 reshape 成 (batch=1, seq_len, embedding_dim) 的序列张量。

- output 是 LSTM 对每个时间步产生的隐状态序列，形状为 (1, seq_len, lstm_hidden_dim) 。

- 代码随后执行 output.contiguous().view(-1, self.lstm_dim) ，把所有时间步“摊平”为 (seq_len, lstm_hidden_dim) ，这样就能对每个时间步分别做分类预测（预测下一个字）。

- 接着 self.fc 把 lstm_hidden_dim → vocab_len ，得到每个时间步对词表中每个字的打分（logits）。

- 最后 LogSoftmax 输出 log-probabilities，配合训练时使用的 NLLLoss（负对数似然损失），构成标准的“分类训练”闭环。

1.3 main.py

在 main.py 我们做了几类关键修复/增强：

- 模块名修复：原脚本中使用 rnn_lstm.xxx ，但实际导入是 import rnn ，因此需要统一改为 rnn.xxx ，否则会 NameError 。

- 梯度裁剪 API ：将 torch.nn.utils.clip_grad_norm 改为 torch.nn.utils.clip_grad_norm_ 。带下划线版本是原地修改梯度，属于 PyTorch 新版常用写法，避免版本差异与警告。

- 数据文件路径修复：将 ./poems.txt 改为基于脚本文件位置的绝对路径，例如 os.path.join(os.path.dirname(os.path.abspath(__file__)), 'poems.txt') 。这样无论在终端当前目录在哪，程序都能正确找到与 main.py 同目录下的 poems.txt 。

- GPU 支持 ：增加 device = torch.device("cuda" if torch.cuda.is_available() else "cpu") ，并将模型与输入张量 .to(device) 。这能让训练/推理在 CUDA 可用时自动使用 GPU，否则回退到 CPU，保证兼容性。

- 生成开头词满足题目要求 ：将最终生成测试改为“日、红、山、夜、湖、海、月”作为 begin word。

2 RNN、LSTM、GRU 模型原理解释

2.1 RNN（循环神经网络）

RNN 专门用于处理序列数据（如文本、语音、时间序列等）。它的核心思想是：网络不仅接收当前时刻的输入，还会接收上一时刻的“隐藏状态”（即过去的记忆）。

核心公式（纯文本表示）：

- $h_t = \tanh(W_x * x_t + W_h * h_{t-1} + b)$

- $y_t = Wy * h_t + c$

(注：h_t 是当前隐藏状态，x_t 是当前输入，h_{t-1} 是上一步的隐藏状态，W 和 b 是权重和偏置。)

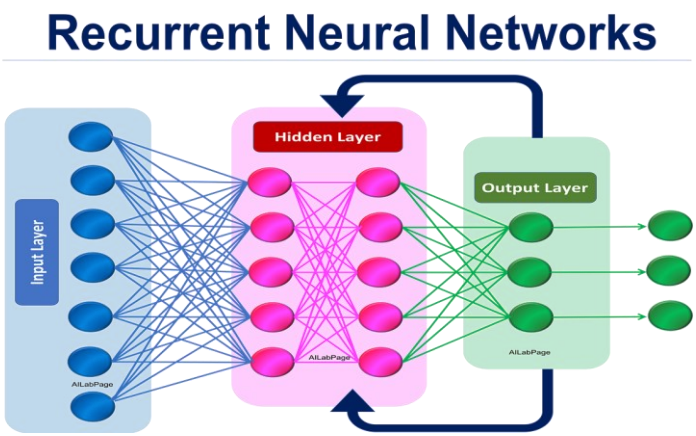

图 1.RNN 模型图

优点： 天然适配序列数据，能够利用历史信息来预测下一步（例如本实验中的“根据前面的字预测下一个字”）。

缺点： 在处理长序列时，由于采用 BPTT 算法，RNN 极易出现“梯度消失”或“梯度爆炸”现象。这导致模型很难记住较早前的信息，难以捕捉“长距离依赖”（例如诗句中跨越多句的对仗或押韵规则）。

2.2 LSTM（长短期记忆网络）

LSTM 是 RNN 的一种改进版本，专门为了解决“长距离依赖”和“梯度消失”问题而设计。它在 RNN 的基础上增加了一个“细胞状态（Cell State）”作为主线记忆，并引入了三个“门（Gate）”来控制信息的流入与流出。

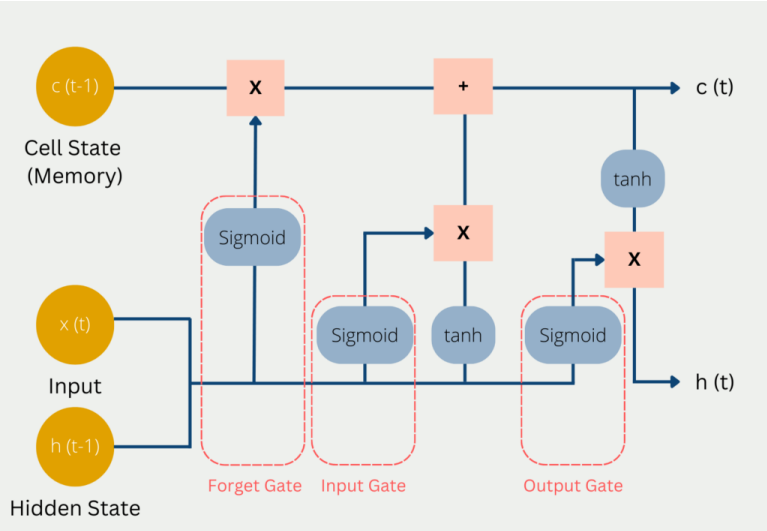

图 2.LSTM 模型图

核心机制与公式：

1. 遗忘门（Forget Gate）：决定要从过去的记忆中丢弃多少信息。

- $f_t = \text{sigmoid}(W_f * [h_{t-1}, x_t] + b_f)$

2. 输入门（Input Gate）：决定要将多少新的输入信息存入当前的细胞状态。

- $i_t = \text{sigmoid}(W_i * [h_{t-1}, x_t] + b_i)$
- $C\sim_t = \tanh(W_c * [h_{t-1}, x_t] + b_c)$ (候选细胞状态)

3. 细胞状态更新（Cell State Update）：结合遗忘门和输入门，更新主线记忆。

- $C_t = f_t * C_{t-1} + i_t * C\sim_t$

4. 输出门（Output Gate）：决定当前细胞状态有多少部分需要输出给隐藏状态。

- $o_t = \text{sigmoid}(W_o * [h_{t-1}, x_t] + b_o)$
- $h_t = o_t * \tanh(C_t)$

在诗歌生成中的优势： 诗歌生成不仅需要局部搭配（相邻字词连贯），还需要较长跨度的结构信息（句式、节奏、主题）。LSTM 的门控机制能够像传送带一样稳定地将重要信息传递到序列末尾，使模型更容易学到诗歌的“整体风格与结构”。

2.3 GRU（门控循环单元）

GRU 是 LSTM 的一种流行变体（简化版），它将 LSTM 的三个门简化为了两个门，并将细胞状态（C_t）与隐藏状态（h_t）合并，用更少的参数实现了类似的效果。

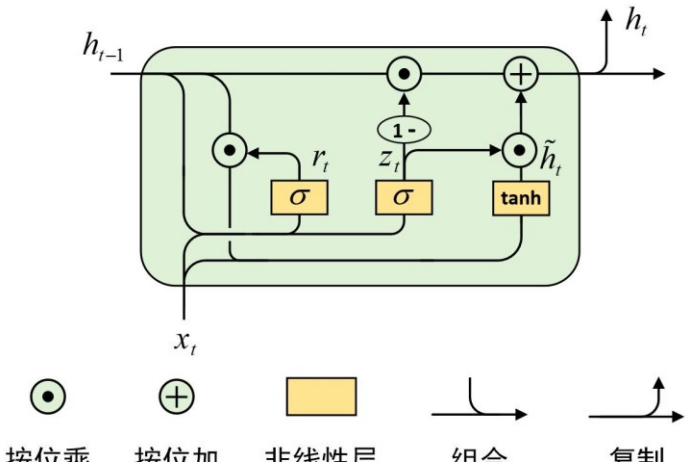

图 3.GRU 模型图

核心机制与公式：

1. 更新门（Update Gate）：类似于 LSTM 中遗忘门和输入门的结合，决定保留多少旧状态以及引入多少新状态。

- $z_t = \text{sigmoid}(W_z * [x_t, h_{t-1}])$

2. 重置门（Reset Gate）：决定在计算新的候选状态时，要忽略多少过去的记忆。

- $r_t = \text{sigmoid}(W_r * [x_t, h_{t-1}])$

3. 候选隐藏状态：

- $h\sim_t = \tanh(W * [x_t, r_t * h_{t-1}])$

4. 最终隐藏状态更新：

- $h_t = (1 - z_t) * h_{t-1} + z_t * h\sim_t$

对比：

GRU 结构更轻量，参数更少，训练速度通常比 LSTM 更快，且在数据量较小时不容易过拟合；LSTM 表达能力更强，在处理极其复杂的长序列依赖时，往往上限更高。在实际应用中，两者的表现通常非常接近。

3 诗歌生成任务的完整流程叙述

3.1 数据预处理与表示

① 从 poems.txt 中逐行读取诗歌，过滤包含特殊字符或过长/过短的样本（保证训练样本质量）。

② 给每首诗添加起始符 G 与终止符 E ：

- G 让模型学习“从开始如何起句”；
- E 让模型学会“什么时候应该停止”。

③ 统计所有字的频次，建立词表 word_int_map ：每个汉字映射到一个整数 ID。

④ 将每首诗转换为 ID 序列（例如 [G, 春, 风, …, E] → [12, 305, 98, …, 7] ）

3.2 Batch 构造与监督信号（teacher forcing）

训练时的监督目标是“下一个字预测”：

- 输入序列： x = [w1, w2, ..., wT]
- 标签序列： y = [w2, w3, ..., wT, wT] （代码里通过平移构造）

这等价于让模型在每个时间步都学习：给定前文，预测下一个字的概率分布。

3.3 模型结构（Embedding → LSTM → Linear → LogSoftmax）

① Embedding ：把离散 ID 映射成稠密向量，使相似字在向量空间中更接近，便于模型学习语义关联与搭配规律。

② 两层 LSTM ：提取序列特征，编码上下文与“风格/节奏”信息。

③ Linear 分类层 ：把隐藏表示映射回词表大小，得到对每个候选字的打分。

④ LogSoftmax + NLLLoss ：对分类打分变成 log 概率并计算损失，优化“真实下一个字”的对数似然。

3.4 推理生成（从 begin word 自回归生成）

生成阶段是典型的自回归过程：

① 给定开头字 begin_word ，初始化 poem = begin_word 。

② 循环执行：

- 将当前 poem 转成 ID 张量输入模型；
- 取模型最后一个时间步的输出（代表“下一字分布”）；
- 通过 argmax 选择概率最大字作为下一个字；
- 拼接到 poem 后继续；

③ 直到生成 E 或达到长度上限（代码里限制 len(poem) > 30 防止死循环）。

4 按指定 begin word 生成诗歌与截图说明

4.1 实验截图

```
******************************
epoch   29 batch number 344 loss is:  6.036376953125
prediction [10, 16, 117, 97, 159, 0, 10, 262, 91, 102, 127, 1, 114, 17, 22, 62, 29, 0, 6, 78, 16, 145, 325, 1, 6, 32, 35,
4, 383, 0, 6, 16, 34, 28, 40, 1, 72, 48, 28, 9, 64, 0, 5, 21, 19, 19, 262, 1, 4, 50, 47, 4, 383, 0, 22, 18, 61, 22, 594, 1
, 3, 17, 9, 155, 74, 0, 6, 24, 4, 360, 121, 1, 3, 3]
b_y        [196, 330, 69, 44, 70, 0, 581, 451, 18, 714, 457, 1, 8, 169, 5, 24, 73, 0, 6, 40, 100, 54, 626, 1, 185, 11, 32,
 1148, 1148, 0, 249, 16, 63, 236, 236, 1, 118, 48, 168, 450, 378, 0, 6, 21, 101, 289, 615, 1, 22, 50, 259, 232, 252, 0, 33
, 364, 117, 93, 594, 1, 233, 161, 88, 201, 943, 0, 86, 30, 43, 39, 632, 1, 3, 3]
******************************
epoch   29 batch number 345 loss is:  6.148830890655518
prediction [10, 48, 19, 10, 70, 0, 10, 41, 4, 30, 20, 1, 10, 117, 117, 4, 39, 0, 10, 117, 10, 147, 21, 1, 10, 48, 28, 6, 1
6, 0, 6, 132, 35, 449, 12, 1, 6, 16, 10, 30, 57, 0, 10, 178, 25, 17, 188, 1, 10, 172, 10, 190, 32, 0, 4, 43, 10, 187, 21,
1, 3, 488, 9, 26, 104, 0, 42, 0, 4, 178, 10, 153, 86, 5, 5, 1, 3, 3]
b_y        [457, 152, 475, 147, 70, 0, 223, 8, 37, 313, 1117, 1, 302, 457, 231, 652, 1824, 0, 222, 704, 70, 147, 5, 1, 457
, 48, 84, 88, 16, 0, 201, 72, 1766, 372, 1209, 1, 88, 16, 112, 594, 93, 0, 12, 57, 56, 23, 79, 1, 149, 278, 80, 309, 2542,
 0, 28, 175, 254, 217, 1113, 1, 271, 488, 450, 135, 43, 15, 42, 0, 12, 65, 564, 475, 540, 157, 29, 1, 3, 3]
******************************
epoch   29 batch number 346 loss is:  6.174585342407227
prediction [10, 23, 28, 6, 48, 0, 16, 0, 84, 178, 10, 77, 97, 69, 102, 1, 10, 559, 97, 0, 69, 237, 6, 0, 10, 1125, 4, 268,
 117, 97, 44, 1, 10, 58, 4, 115, 22, 149, 51, 0, 5, 11, 68, 48, 4, 26, 446, 1, 4, 5, 4, 30, 4, 147, 85, 0, 10, 117, 10, 16
3, 4, 4, 1, 1, 3, 30, 4, 11, 695, 108, 0, 0, 4, 131, 4, 18, 69, 6, 131, 1, 3, 3]
b_y        [188, 682, 804, 242, 143, 242, 70, 0, 10, 242, 297, 41, 10, 242, 100, 1, 342, 71, 191, 6, 240, 60, 154, 0, 555,
 85, 1335, 1335, 69, 97, 107, 1, 139, 284, 325, 196, 4, 149, 95, 0, 36, 11, 99, 73, 144, 37, 381, 1, 38, 5, 112, 163, 1572
, 241, 867, 0, 182, 7, 1076, 1076, 243, 134, 434, 1, 441, 70, 296, 457, 869, 108, 20, 0, 729, 131, 907, 117, 66, 6, 539, 1
, 3, 3]
******************************
```

图 4.训练过程截图

```
******************************
epoch   29 batch number 347 loss is:  6.1374430656433105
error
inital  linear weight
日离一年少，一夜月明看远楼。
一夜不能生海水，一生无事在东京。
error
inital  linear weight
红树下双泪，一望千秋万六年。
一夜不能生海水，一生无事在东京。
error
inital  linear weight
山下草，白云生处处时来。
不如山下无人识，不得南山有一声。
error
inital  linear weight
一望一千年，一枝无一事。
error
inital  linear weight
湖月夜长夜，夜夜秋云入夜流。
不见东阳归不得，不能生处有无声。
error
inital  linear weight
一枝不可如相问，一望无人更不回。
```

图 5.生成结果截图

4.2 实验总结

①本实验本质是“字符级语言模型”，通过 LSTM 学习古诗语料的统计规律与序列模式，实现从给定开头字出发的逐字生成。

②两层 LSTM 相比简单 RNN 更能捕捉长距离依赖，训练更稳定，生成的整体连贯性通常更好。

③训练时使用梯度裁剪可以缓解梯度爆炸，配合合适的学习率与优化器能提升收敛稳定性。

④运行环境上，若使用 GPU（CUDA 可用），训练速度会明显提升。